Phishing URL Detection Through Structural-Group Evaluation, Model Optimization, and Ensemble Learning

***Abstract*—Phishing URLs are a persistent threat to online communication and financial services. Although machine-learning classifiers often report extremely high accuracy on phishing benchmarks, those scores may not fully characterize performance under a more restrictive separation of related URL structures. This paper presents an experimental comparison of five supervised classifiers—logistic regression, support vector machine (SVM), decision tree, random forest, and XGBoost—alongside stacking and blending ensembles. The experiments use a URL-feature CSV containing 101,219 initial records and 16 engineered numeric predictors. After duplicate removal and elimination of ambiguous feature vectors, 31,808 records remain. A group-disjoint holdout contains 27,559 training and 4,249 test records. On this holdout, the highest recorded baseline accuracy is 99.72% for XGBoost, with phishing-class F1 of 99.82%. Hyperparameter search optimized for phishing recall does not improve all test metrics: the tuned SVM obtains 98.59% accuracy and a phishing-class F1 of 99.09%, while stacking and blending achieve 97.79% and 97.95% accuracy, respectively. Random oversampling, SMOTE, and ADASYN are evaluated as additional training-only comparisons. Compared with the publicly accessible URLNet research by Le et al., this study emphasizes engineered numerical URL features, structural-group testing, recall-oriented tuning, stacking versus blending, and oversampling ablations rather than end-to-end convolutional representation learning. The studies use different datasets, label definitions, and evaluation protocols; no cross-paper accuracy superiority is claimed. The findings show the strength of an XGBoost baseline and the importance of reporting unfavorable outcomes rather than assuming that more complex ensembles generalize better.**

***Index Terms*—phishing URL detection, machine learning, XGBoost, support vector machine, stacking, blending, class imbalance, group-disjoint evaluation**

I. Introduction

Phishing websites attempt to deceive users into disclosing sensitive information by imitating trusted services or embedding malicious destinations in apparently legitimate links. Automatically classifying suspicious URLs is therefore a relevant cybersecurity task. Machine-learning methods can learn patterns in lexical and structural URL attributes, including URL length, punctuation, domain characteristics, and character distributions. However, a high score on one randomly partitioned dataset should not automatically be interpreted as reliable detection of novel phishing campaigns.

A well-established publicly accessible research reference is URLNet by Le et al. [1]. URLNet learns URL representations directly from character and word sequences using convolutional neural networks and was evaluated at substantially larger scale than the present experiment. It reported ROC-AUC of 0.9929 for its full model trained on five million examples and tested on a corpus of ten million URLs. Its research question concerns learned lexical representations and their ability to outperform expert-engineered alternatives. The present work instead examines evaluation design and training choices for pre-extracted numerical features. Accordingly, a numerical contest between the two papers would be invalid without a shared dataset and test protocol.

The present study asks three questions: (i) how do five conventional models perform when structural feature groups are held out together; (ii) does tuning for phishing recall consistently improve accuracy and F1; and (iii) do stacking, blending, and oversampling outperform a strong standalone baseline? The main contribution is an *empirical evaluation design*, not a new classifier architecture. Specifically, the notebook implements a structural-group split, a fixed evaluation suite emphasizing phishing-class performance, independent model ranking for an ensemble, and three resampling comparisons. It also exposes cases in which tuning and ensembling underperform a baseline.

II. Related Work and Research Gap

A. URLNet: Established Research Baseline

Le et al. proposed URLNet as an end-to-end framework for malicious URL detection [1]. Its full architecture integrates character-level CNNs, word-level CNNs, and character-aware word embeddings to learn discriminative URL patterns without relying solely on manually designed lexical predictors. The original study provides experiments across different training-corpus sizes, comparisons with bag-of-words and character-trigram baselines, component ablations, and low-false-positive-rate analyses. Its evaluation specifically includes ROC-AUC and true-positive rate at selected false-positive-rate thresholds. The published full URLNet model reaches AUC 0.9885 with one million training samples and 0.9929 with five million, using a ten-million-URL test corpus. The reported task includes malicious URLs more broadly and therefore is not an identical phishing-only benchmark.

B. Complementary Evaluation Questions

URLNet's major contribution is *representation learning*: capturing ordered lexical information and reducing reliance on manually engineered feature sets. Our experiment does *not* resolve URLNet's stated representation-learning problem, and it does not reproduce the URLNet architecture. Instead it addresses complementary questions that are not directly

explored in URLNet's reported experiments: (i) holding out constructed groups of similar numerical URL structures; (ii) checking whether optimizing phishing recall harms balanced accuracy; (iii) directly comparing conventional stacking and blending on fixed numeric predictors; and (iv) measuring how random oversampling, SMOTE, and ADASYN change results under a common holdout. These are differences in experimental scope, *not verified deficiencies* of URLNet or proof that our models generalize better.

Research on resampling methods motivates the augmentation comparison [2], [3]. Importantly, URLNet already includes its own extensive architecture ablations and low-FPR operating-point evaluation, which the present notebook does not reproduce. The studies should be viewed as complementary rather than competing leader-board entries.

III. Materials and Methods

A. Dataset and Feature Preparation

The experiment loads a locally uploaded file named `url_features_extracted.csv`. Its original matrix has 101,219 rows and 18 columns, including the URL field and binary `ClassLabel`. The notebook reports 346 exact duplicate rows and one missing target. Records without a valid binary label are excluded. Numeric conversion and duplicate removal are followed by elimination of identical predictor vectors associated with conflicting class labels and deduplication on predictors plus target. This pipeline produces 31,808 usable rows. The exact original data source, collection time, and label-validation protocol are not specified in the notebook; consequently, they cannot be independently claimed here.

The 16 features are `url_length`, `has_ip_address`, `dot_count`, `https_flag`, `url_entropy`, `token_count`, `subdomain_count`, `query_param_count`, `tld_length`, `path_length`, `has_hyphen_in_domain`, `number_of_digits`, `tld_popularity`, `suspicious_file_extension`, `domain_name_length`, and `percentage_numeric_chars`. Class 0 represents phishing and class 1 represents legitimate URLs in the notebook's evaluation code. The cleaned dataset contains 77.81% class 0 and 22.19% class 1, making phishing the majority class in this particular dataset.

Figure 1 summarizes the notebook-based experimental design.

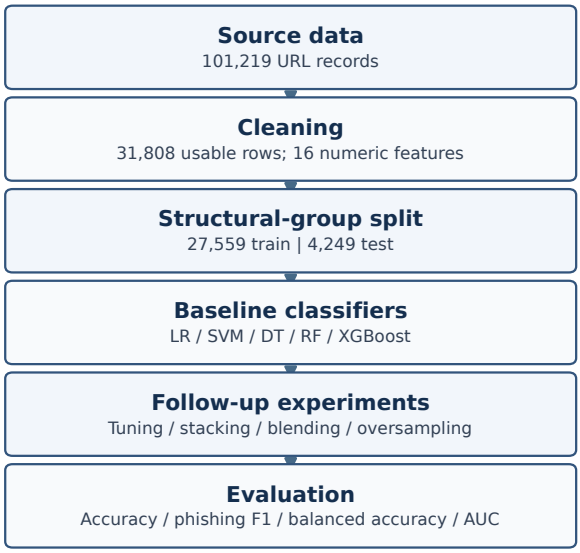


Fig. 1. Experimental workflow reconstructed from executed Colab notebook stages. Tuning, ensembling, and resampling are separate follow-up comparisons rather than one sequential final model.

B. Structural-Group Holdout

A deterministic group identifier is assembled from four binary-like fields and quantile-binned numeric features. `GroupShuffleSplit` then allocates approximately 20% of records for testing, retaining all records with the same constructed structural signature on one side of the split. The realized training and test sizes are 27,559 and 4,249, and the notebook reports zero overlap in constructed group identifiers. Training and test phishing proportions are 77.79% and 77.92%, respectively. Median imputation is fitted to the training partition and applied to both partitions; standardized predictors are used within the logistic-regression and SVM pipelines.

This protocol is stricter than an ordinary record-level random split with respect to the chosen structural signatures. Nevertheless, it does *not* demonstrate a domain-disjoint, time-disjoint, or externally validated split. Because group identifiers are constructed from numerical feature bins rather than real-world attack-campaign IDs, the experiment should be interpreted as a *structural-group* robustness check only.

C. Classifiers and Tuning

Five regularized or capacity-constrained initial classifiers are evaluated without oversampling: logistic regression, SVM, decision tree, random forest, and XGBoost. Their hyperparameters are subsequently searched using `RandomizedSearchCV`. The optimization score is phishing recall, defined with class 0 as the positive class:

$$R_{\text{phish}} = \frac{TP_0}{TP_0 + FN_0}. \tag{1}$$

Logistic regression, decision tree, random forest, and XGBoost use five-fold cross-validation during search; SVM uses three-fold search on a sample of at most 25,000 training records. The tuned XGBoost model is separately fitted with an internal training/validation split and early stopping. These choices make the tuned-vs-baseline comparison a comparison of complete training procedures, not solely a comparison of hyperparameter values.

A distinct 20% split within the training set ranks the tuned models by balanced accuracy, phishing F1, and phishing recall. SVM ranks first and is selected as the meta-learner; logistic regression, decision tree, and random forest become the

TABLE I
INITIAL MODEL RESULTS ON THE STRUCTURAL-GROUP TEST SET (%)

Model	Acc.	Bal. Acc.	Phish F1	AUC
Logistic regression	97.79	98.05	98.57	99.47
SVM	98.59	98.75	99.09	99.75
Decision tree	99.55	99.41	99.71	99.91
Random forest	99.62	99.15	99.76	99.96
XGBoost	**99.72**	99.44	**99.82**	99.96

three base learners. Stacking uses cross-validated base-model probabilities with a three-fold stacking procedure; blending trains base learners on 80% of the training data and fits the meta-learner on the remaining 20%. No test labels are used to select these ensemble roles.

D. Oversampling and Evaluation

A separate experiment trains the selected SVM with random oversampling, SMOTE, and ADASYN. Resampling is applied only to the training partition before fitting, leaving the structural-group test set unchanged. The notebook records accuracy, balanced accuracy, phishing precision, phishing recall, phishing F1, Matthews correlation coefficient (MCC), and ROC AUC. For phishing designated as class 0,

$$F1_{\text{phish}} = \frac{2P_{\text{phish}}R_{\text{phish}}}{P_{\text{phish}} + R_{\text{phish}}}. \tag{2}$$

AUC calculations consistently treat class 0 as the positive class. The notebook also produces class-distribution and feature-correlation visualizations. Despite mentions of NLP, large language models, and explainable AI in notebook headings/imports, the executed cells do not contain completed TF–IDF, Word2Vec, FastText, BERT, GPT, Grok, or LIME experiments; no results for these methods are claimed.

IV. RESULTS AND DISCUSSION

A. Five-Classifier Baselines

Table I and Fig. 2 present the five initial-model outcomes from the saved notebook outputs. XGBoost has the highest recorded accuracy (99.72%) and phishing F1 (99.82%), followed closely by random forest. The baseline decision tree also performs strongly. Thus, this dataset does not require an ensemble to obtain its best observed accuracy.

B. Tuned Models and Ensembles

As shown in Table II and Fig. 3, tuning for recall does not yield an across-the-board improvement. Although the tuned decision tree, random forest, XGBoost, and stacking each record phishing recall of 100%, their accuracies are 97.79% and their balanced accuracies are 94.99%. This behavior is consistent with predictions that capture all phishing cases but incorrectly flag a subset of legitimate cases. In contrast, the tuned SVM reaches 98.59% accuracy with 98.49% phishing recall, while the tuned logistic regression reaches 98.00% accuracy. The strongest *recorded overall test accuracy* remains the initial XGBoost result rather than a tuned or ensemble result.

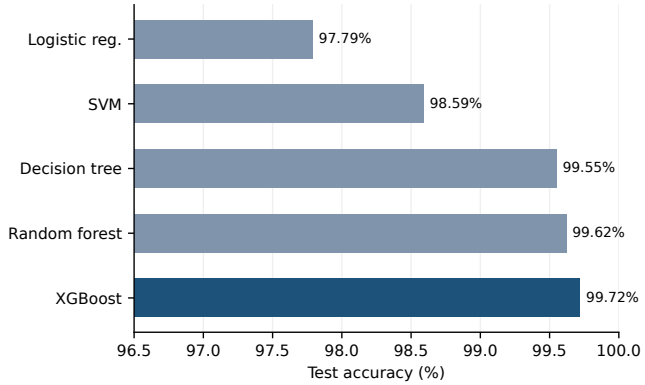


Fig. 2. Test accuracy of the five initial classifiers on the structural-group holdout. Values are taken from the notebook's saved evaluation outputs; XGBoost is the strongest recorded baseline.

TABLE II
TUNED AND ENSEMBLE TEST RESULTS (%)

Model	Acc.	Bal. Acc.	Phish recall	Phish F1
Tuned logistic reg.	98.00	98.22	97.83	98.70
Tuned SVM	98.59	98.71	98.49	99.09
Tuned decision tree	97.79	94.99	100.00	98.60
Tuned random forest	97.79	94.99	100.00	98.60
Tuned XGBoost	97.79	94.99	100.00	98.60
Stacking	97.79	94.99	100.00	98.60
Blending	97.95	97.08	98.64	98.69

Blending has slightly higher accuracy (97.95%) and balanced accuracy (97.08%) than stacking (97.79% and 94.99%). Stacking maximizes phishing recall on the reported test set but gives a lower balanced accuracy, illustrating a consequential trade-off when legitimate URLs are incorrectly classified as phishing. The notebook times the stacking fit at about 31.28 seconds but measures only the blending *meta-learner fit* at 0.13 seconds. Accordingly, these timing values must *not* be interpreted as a fair end-to-end comparison of computational cost.

C. Augmentation Ablation

Table III shows resampling results for the selected SVM. SMOTE obtains the highest accuracy and phishing F1 among the three augmented variants, although both are below the unaugmented tuned SVM's 98.59% accuracy and 99.09% phishing F1. Random oversampling has the highest balanced accuracy of the three, while ADASYN performs least favorably. Because phishing is already the majority class and the evaluation uses phishing as the positive class, balancing the minority *legitimate* class can change the trade-off in a way that does not improve phishing F1. The comparison demonstrates why the effect of resampling should be verified rather than assumed.

D. Comparison with Publicly Available URLNet Research

Table IV compares the present notebook with the published URLNet work [1]. URLNet uses raw URL strings and CNN

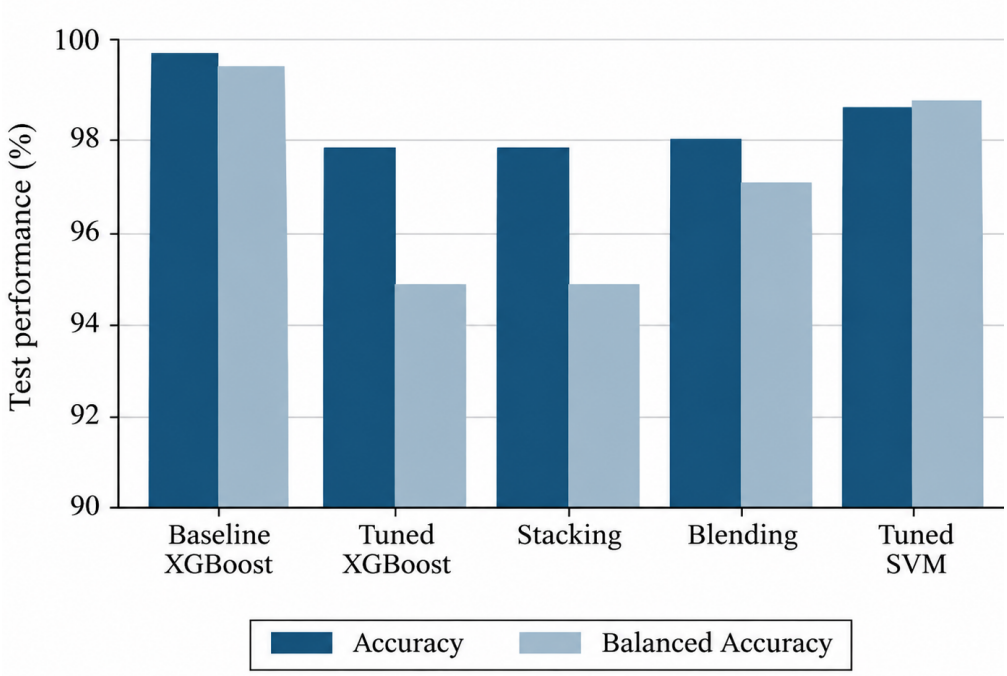


Fig. 3. Overall accuracy versus balanced accuracy for selected baseline, tuned, and ensemble configurations. The discrepancy illustrates the trade-off introduced by recall-oriented optimization.

TABLE III
TRAINING-ONLY OVERSAMPLING WITH TUNED SVM (%)

Method	Acc.	Bal. Acc.	Phish recall	Phish F1
Random oversampling	97.93	98.56	97.43	98.65
SMOTE	97.95	98.53	97.49	98.67
ADASYN	97.51	98.28	96.89	98.37

TABLE IV
STUDY DESIGN COMPARISON: URLNET VS. THE PRESENT WORK

Aspect	URLNet [1]	This notebook
Input	Raw URL text; character and word sequences	16 engineered numeric URL features
Architecture	Joint character/word CNN with embeddings	Five conventional ML models; stacking and blending
Scale	1m/5m training; 10m test examples	27,559 training; 4,249 test records
Metrics	ROC-AUC and TPR at prescribed FPR levels	Accuracy, balanced accuracy, phishing F1/recall, MCC, AUC
Evaluation	Large-scale held-out test corpus	Constructed structural-group holdout
Ablations	Network components and training scale	Tuning, ensembles and three oversamplers
Reported result	Full-model AUC 0.9929 (5m training)	Baseline XGBoost AUC approx. 0.9996; accuracy 99.72%

embeddings, whereas our notebook uses 16 engineered numeric features. URLNet evaluates very large-scale malicious URL classification and reports AUC and true-positive rate at specific false-positive-rate levels; our notebook evaluates a cleaned phishing-oriented binary dataset using a structural-group holdout and class-specific metrics. URLNet already provides architecture ablation studies, whereas our ablations concern training procedures and resampling.

The last row is *descriptive, not a head-to-head ranking*: AUC values on different datasets cannot establish superiority, and URLNet's task includes malicious URL categories beyond phishing. The notebook's additional methodological checks may help characterize engineered-feature classifier behavior under its own distribution shift, but no experiments test whether they fix a demonstrated failure of URLNet. A conclusive comparison would require training both models on identical data, matching labels and splits, and evaluating at common false-positive-rate thresholds.

E. Validity and Limitations

Several boundaries are important. First, the notebook does not document where the uploaded CSV originated or how its labels were verified. Second, the structural groups are based on binned predictor values; they do not guarantee disjoint URLs by registrable domain, campaign, or collection date. Third, only one group-disjoint holdout and one set of saved model results are presented, without confidence intervals or statistical significance tests. Fourth, the entire cleaned dataset is used in constructing quantile bins for group definition; although no labels are used in this grouping operation, future work should explicitly document how grouping choices are fixed. Fifth, the model-ranking protocol and multi-model tuning involve internal splits, but a separate external benchmark is still needed before making generalization claims. Finally, a last notebook

cell refers to an undefined `best_xai_model` variable, so its displayed confusion-matrix graphic cannot be treated as a verified, reproducible final-model output. No explainability or NLP experiment is reported as completed.

V. Conclusion and Future Work

This paper documents a reproducible-in-principle comparison of URL-feature classifiers, ensemble strategies, and imbalance treatments from the supplied Colab notebook. A structural-group holdout helps test extrapolation to unseen combinations of engineered URL features. The initial XGBoost classifier attains the highest recorded test accuracy, 99.72%, with phishing F1 of 99.82%. In contrast, recall-oriented tuning, stacking, blending, and the examined resampling techniques do not surpass that baseline on the same held-out set. Relative to URLNet, the distinguishing contribution is a complementary evaluation of conventional classifiers using numerical predictors, structural grouping, ensemble choices, and resampling; no superiority over URLNet is established.

Future research should document dataset provenance, repeat the evaluation across multiple group splits, introduce genuine domain- and time-disjoint benchmarks, report confidence intervals and low-FPR detection performance, and measure full-pipeline inference latency. A rigorous future comparison should reimplement URLNet on the identical labeled corpus and split. Separate executed experiments would be required before asserting results for text embeddings, transformer/LLM baselines, or LIME/SHAP explainability. These steps would make claims about practical phishing detection more defensible.

References

[1] H. Le, Q. Pham, D. Sahoo, and S. C. H. Hoi, "URLNet: Learning a URL representation with deep learning for malicious URL detection," 2018, arXiv:1802.03162. [Online]. Available: https://arxiv.org/pdf/1802.03162. University repository: https://ink.library.smu.edu.sg/sis_research/4135/.

[2] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[3] H. He, Y. Bai, E. A. Garcia, and S. Li, "ADASYN: Adaptive synthetic sampling approach for imbalanced learning," in *Proc. IEEE Int. Joint Conf. Neural Networks*, 2008, pp. 1322–1328.